

Attention Is All You Need

Vaswani et al., 2017 / 24기 김지원 / 6주차 논문 정리

Abstract

원래 번역 같은 문장 처리는 RNN(순환신경망)처럼 단어를 하나씩 순서대로 읽는 모델이 주류였는데, 이 논문은 순환구조(RNN)랑 합성곱(CNN)을 다 빼고 "어텐션(attention)"만으로 Transformer라는 새 모델을 제안. 순서대로 안 읽고 병렬 계산이라 학습 훨씬 빠르는데, 번역 성능(BLEU)은 오히려 기존 최고 기록보다 높게 나옴.

1 Introduction

RNN은 문장을 앞에서부터 한 단어씩 순서대로 봐야 해서 길어질수록 병렬 계산이 어렵고 느림. 하지만 이 논문은 순서대로 읽는 걸 버리고, 문장 안 단어들 관련성을 한 번에 계산하는 "어텐션"만으로 모델 구성. GPU 8개로 12시간만 학습해도 기존 최고 성능 넘김 학습 시간 짧은 게 인상적.

2 Background

이전에도 계산 병렬화 시도(합성곱 기반 모델 등)는 있었는데, 문장에서 멀리 떨어진 단어 둘을 연결하려면 여러 층을 거쳐야 해서 여전히 느렸음. **Transformer**는 어텐션만으로 문장 내 어떤 두 단어든 계산 한 번에 연결 가능. 이 문제를 해결한 게 핵심.

3 Model Architecture

Transformer도 다른 번역 모델처럼 입력을 읽는 인코더(Encoder)와, 그 정보로 출력을 한 단어씩 만드는 디코더(Decoder)로 구성. 차이점은 내부가 RNN이 아니라 전부 어텐션과 완전연결층(fully-connected layer)만으로 이루어져 있다는 것.

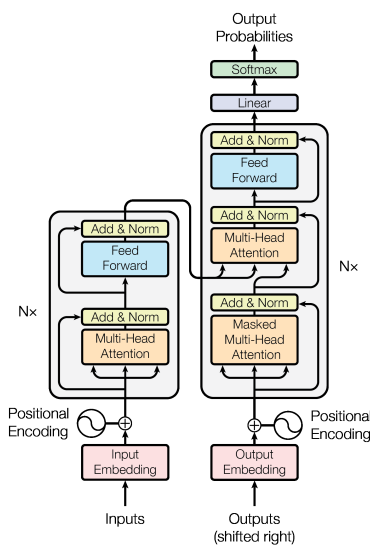


그림 1: Transformer 전체 구조 -- 왼쪽이 인코더(입력 처리), 오른쪽이 디코더(출력 생성). 둘 다 이 블록을 $N=6$ 번 쌓는다.

3.1 Encoder and Decoder Stacks

인코더는 같은 구조의 층을 6개 쌓은 것. 층마다 (1) 단어끼리 서로 참고하는 멀티헤드 어텐션, (2) 위치별로 적용되는 완전연결층을 거치고, 각 단계 뒤엔 잔차연결과 정규화 추가. 디코더도 구조 비슷한데 인코더 정보를 참고하는 어텐션이 하나 더 있고, 미래 단어 못 보게 가리는(masking) 장치 추가.

3.2 Attention

어텐션 = "질문(Query)"이 주어졌을 때 여러 "키(Key)-값(Value)" 쌍 중 어디에 집중할지 계산해서, 값들을 가중평균해 결과 하나로 뽑는 연산. 한 줄 요약: "이 단어를 이해하려면 문장 어디를 더 봐야 하는지"를 자동으로 찾아주는 것.

3.2.1 Scaled Dot-Product Attention

Query와 Key를 내적해서 유사도 구하고, Key 차원 수의 제곱근으로 나눠 값이 너무 커지지 않게 조정 후, softmax로 가중치 만들어 Value에 곱해 더함. 이 스케일링 안 하면 값이 너무 커져서 학습이 잘 안 됐다고 함. 이유는 각주에 설명.

3.2.2 Multi-Head Attention

어텐션 한 번만 크게 하지 않고, Query/Key/Value를 8개의 작은 조각으로 나눠 각자 어텐션 계산한 뒤 다시 이어붙임. "문법", "의미"처럼 서로 다른 관련성을 각 헤드가 따로 학습 가능 한 번만 크게 하는 것보다 정보 풍부하게 추출.

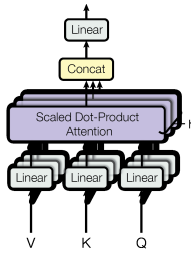


그림 2: Multi-Head Attention -- Q,K,V를 여러 갈래로 나눠 각각 어텐션을 계산하고(Scaled Dot-Product Attention을 h번 병렬로), 다시 합친다(Concat).

3.2.3 Applications of Attention in our Model

Transformer는 어텐션을 세 군데에 사용: (1) 디코더가 인코더 출력 전체를 참고하는 어텐션, (2) 인코더 안에서 문장 자기 자신을 참고하는 셀프어텐션, (3) 디코더 안 셀프어텐션 단, 아직 안 나온 미래 단어는 못 보게 가려서(masking) 순서 유지.

3.3 Position-wise Feed-Forward Networks

어텐션 층 다음엔 각 단어 위치마다 똑같이 적용되는 2층짜리 작은 신경망(선형변환 -> ReLU -> 선형변환) 하나씩 붙음. 커널 크기 1인 합성곱 두 개 이어붙인 것과 동일하다고 봄.

3.4 Embeddings and Softmax

단어를 숫자 벡터로 바꾸는 임베딩 층과, 다음 단어 예측용 선형변환+softmax 층에서 같은 가중치 행렬 공유. 파라미터 수 절약 목적.

3.5 Positional Encoding

Transformer는 RNN과 달리 단어를 순서대로 안 읽어서 순서 정보를 따로 넣어줘야 함. 사인/코사인 함수로 만든 "위치 인코딩" 값을 단어 임베딩에 더해줌. 학습으로 위치를 따로 배우게 하는 것도 실험했는데 결과 거의 동일.

4 Why Self-Attention

셀프어텐션을 계산량, 병렬처리 가능 정도, 멀리 떨어진 두 단어 연결하는 "경로 길이" 세 기준으로 RNN·합성곱과 비교. 문장 길이가 벡터 차원보다 작은 대부분 경우 셀프어텐션이 가장 빠르고, 어떤 두 단어든 계산 한 번에 연결돼 장거리 관계 학습에 유리.

5 Training

영어-독일어 450만 쌍, 영어-프랑스어 3,600만 쌍으로 학습(5.1). GPU 8장, base 모델 12시간·큰 모델 3.5일 소요(5.2). Adam 옵티마이저, 학습률 초반 상승 후 하강(5.3). 과적합 방지로 드롭아웃·레이블 스무딩 적용(5.4).

6 Results

영어-독일어 번역에서 기존 최고 기록보다 2점 이상 높은 BLEU 28.4 기록, 학습 비용은 훨씬 적음(6.1). 헤드 개수·모델 크기·드롭아웃 조정 실험 헤드 너무 적거나 많아도 성능 하락, 모델 클수록 좋음(6.2). 번역용 모델을 구문분석 과제에도 적용해봤는데 전문 모델과 비슷하거나 더 좋은 성능 범용성 있다는 뜻(6.3).

7 Conclusion

RNN·합성곱 없이 어텐션만으로 만든 최초의 모델 Transformer 제안 논문. 번역에서 속도·성능 둘 다 기존 모델 압도. 이미지·음성 등 다른 데이터로 확장 계획 밝힘. 이후 이 구조가 BERT, GPT의 기반이 됨 영향력 큰 논문.